

A Statistical Model of Privacy-Budget Consumption in Repeated Aggregate SQL Workloads

CMP653 Project Final Report

Refik Can Öztaş
Hacettepe University
Ankara, Turkey
N25142279

Abstract

Practical SQL workloads repeat templates—dashboards, drill-downs, and parametric filter sweeps—while most privacy accounting treats queries in isolation. I present an analytical model that relates workload structure (template repetition skew, group cardinality, temporal staleness) to privacy budget consumption and utility, and instantiate it inside a differentially private SQL middleware over PostgreSQL/DuckDB and TPC-H. The model gives a closed-form expression for the expected budget as a product of the per-query budget and the expected unique-template count, with two limits: deterministic repetition reduces to the $1 - 1/k$ savings ratio, and uniform sampling over many templates recovers naive sequential composition. A temporal extension with staleness tolerance τ and Poisson update rate λ couples budget consumption with data freshness. I additionally derive a predictive online allocator that targets the offline-optimal $\epsilon_q^* = B/u_k$ from the model’s runtime estimate of u_k , trading a 5–17% MAE reduction for an explicit 12–31% query-rejection rate when the budget is exhausted. The model is empirically validated on a 4,155-trial campaign covering Zipf-parameterized workloads ($\alpha \in \{0, \dots, 10\}$), six workload families (W1–W4 plus two TPC-H parametric variants), four per-query privacy parameters ($\epsilon_q \in \{0.1, 0.5, 1.0, 2.0\}$), and TPC-H scale factors SF=1 and SF=10 (60M lineitem rows). The model’s predictions match empirical means within $< 3\%$ in 21 of 24 main-grid cells and within 0.034 units across the SF=1/SF=10 cross-scale test. Three leakage experiments—single-query MIA, differencing-style reconstruction on a five-level drill-down, and shadow-model MIA across W1–W4—track tight theoretical bounds and expose slack in the cumulative-budget bound. The middleware is a unit-tested prototype layered over PostgreSQL/DuckDB; exact-repeat caching is treated as a corollary of the model’s $\alpha \rightarrow \infty$ limit rather than as a contribution.

1 Introduction

Repeated analytical workloads couple two deployment quantities in differentially private SQL middleware: consumed privacy budget and expected analyst utility. Both are workload-dependent. A dashboard tile that re-issues an identical query a hundred times has very different privacy economics from a drill-down session that strictly narrows each step. The middleware supports a scoped SQL subset (COUNT, SUM, AVG with simple WHERE and GROUP BY); the contribution is an analytical model that predicts privacy/utility behaviour from workload parameters before any query is issued.

Motivation: aggregates leak. Two overlapping COUNT queries can isolate an individual via the textbook differencing attack [1];

sufficiently many noisy aggregate releases enable full database reconstruction [2]. Differential privacy [3] provides the standard defence by adding calibrated Laplace noise. In a deployment, however, the engineering question is no longer “can I make one query private” but “how does a finite budget ϵ_{tot} degrade as the workload runs?”

What this report contributes. Exact-repeat caching is a mechanism, not the central contribution. Returning a cached noisy result is a direct application of the post-processing property of DP and does not by itself answer *when and by how much* caching helps. I promote the contribution to a model that gives quantitative predictions for arbitrary workload distributions and recovers caching as its $\alpha \rightarrow \infty$ corner case.

The contributions are:

- (1) A statistical model (Section 4) that, given a workload distribution $\{p_i\}$ over query templates and a workload length k , yields closed-form predictions for the expected unique-query count $\mathbb{E}[u_k]$, the workload-aware budget consumption $\mathbb{E}[\epsilon_{\text{wa}}(k)]$, and the budget savings ratio $S(k)$. Five propositions cover (i) $\mathbb{E}[u_k]$, (ii) budget savings, (iii) the Zipf-parameterized case, (iv) concentration via McDiarmid’s inequality, and (v) per-query utility under a fixed total budget.
- (2) A temporal extension (Section 5) that introduces a staleness tolerance τ and a Poisson update rate λ , producing $\mathbb{E}[\epsilon_{\text{temp}}(T)]$ over a time horizon T . The static regime, the bounded-staleness regime, and the update-driven regime are continuous slidings of the same expression.
- (3) A middleware (Section 6) that instantiates the model. The system is implemented in Python on top of DuckDB (with an alternative PostgreSQL backend), supports a single-table aggregate SQL subset, and exposes four execution modes (exact, naive DP, workload-aware DP, temporal DP).
- (4) A predictive online budget allocator (Section 10) that uses the model’s $\mathbb{E}[u_k]$ estimate to choose ϵ_q at runtime, targeting the offline optimum $\epsilon_q^* = B/u_k$. It reduces per-query MAE by 5–17% versus naive composition under the same total budget, at the explicit cost of rejecting 12–31% of late queries once the budget is exhausted.
- (5) Empirical validation (Section 7) on Zipf-parameterized workloads at TPC-H scale factor SF=1 (and a cross-scale check at SF=10 with 60M lineitem rows) plus the UCI Adult dataset. The full experimental campaign comprises six sweeps totaling 4,155 trials. On the main grid ($6 \times 4 = 24$ (α, k) cells,

30 trials each), the model’s prediction of $\mathbb{E}[u_k]$ matches the empirical mean within $< 3\%$ in 21 of 24 cells.

- (6) A leakage analysis (Section 9) that quantifies membership-inference AUC and differencing-attack reconstruction error as a function of ϵ , against the theoretical references $e^\epsilon/(1 + e^\epsilon)$ (an upper bound on attacker accuracy) and $1.5/\epsilon$ (the expected absolute pair-difference) respectively.

Research question (reframed). Given a workload distribution and a privacy budget, can a closed-form statistical model predict the realized budget consumption and per-query utility, and does it explain when workload-aware accounting beats naive composition?

The original milestone research question (“does caching help?”) is recovered as the special case $\alpha \rightarrow \infty$ of the model (Proposition 2, Limit A), where the savings ratio reduces to $1 - 1/k$.

2 Background and Related Work

Differential privacy. ϵ -differential privacy [3, 5] bounds the multiplicative change in output distribution between neighbouring databases. The Laplace mechanism adds noise $\text{Lap}(\Delta f/\epsilon)$ where Δf is the query’s global sensitivity. Composition theorems—basic, advanced [4], and Rényi DP [11]—bound cumulative privacy loss across queries.

Attacks that justify DP. Denning [1] characterized the tracker attack class. Dinur and Nissim [2] proved the *Fundamental Law of Information Recovery*: too many noisy aggregate answers reconstruct the database. NIST SP 800-226 [12] compiles current guidance for evaluating DP guarantees in practice.

Private SQL systems. PINQ [10] introduced privacy tracking in a LINQ-style language. Johnson et al. [7] and PrivateSQL [9] expanded the supported SQL subset; DOP-SQL [13] integrates such mechanisms into PostgreSQL. Chorus [8] rewrites queries to inject DP before execution, keeping the host DBMS unmodified.

Multi-query privacy accounting. Dong, Sun, and Yi [14] showed that multiple relational queries can sometimes be answered with tighter error than naive composition suggests. This work shares the same motivation. The difference: [14] exploits relational structure within a join-heavy workload, whereas the present report exploits *template repetition* within a single-table aggregate workload, which is empirically the more common dashboard / drill-down setting.

Workload-aware DP mechanisms. A second line of work optimizes noise placement across a fixed linear-query workload rather than across SQL templates. The Matrix Mechanism [19] chooses a strategy matrix that minimizes total error for a linear-query workload; HDMM [20] scales this to high-dimensional predicate workloads via implicit workload representations. These methods are workload-aware in the sense of joint noise optimization, but they target static range / counting queries and do not model template repetition or adapt ϵ_q at runtime. PrivateSQL’s view selection over a representative workload [9] is conceptually closer to template tracking but still makes an offline allocation decision.

Online and adaptive DP. Roth and Roughgarden’s median mechanism [22] and Hardt–Rothblum’s Private Multiplicative Weights [23] answer adaptively chosen predicate / counting queries via online

learning, exploiting query correlations to answer exponentially more queries than naive Laplace. MWEM [21] extends this to synthetic-data release via Multiplicative Weights + Exponential Mechanism. APEX [24] reverses the analyst’s interface from ϵ to an accuracy bound and picks the mechanism that meets it at minimum privacy loss. Pythia [25] privately extracts data-dependent features from the sensitive input (spending part of the budget) to select the best DP algorithm for a new task. These systems are adaptive in different senses—query selection (MWEM, PMW), per-query cost translation (APEX), or algorithm selection (Pythia)—but none is driven by an analytical forecast of *future* budget consumption derived from observed template frequencies, and none addresses temporal staleness.

Modern / production DP-SQL. Tumult Analytics [26] is a production-grade Python/Spark framework with multi-table sessions, private joins, and explicit adaptive composition; OpenDP and SmartNoise SQL [27] wrap existing databases for SQL/Pandas/Spark deployment. Wilson et al.’s DP-SQL with Bounded User Contribution [28] introduces row-ownership and contribution-bounding operators in relational algebra, addressing user-level DP for production engines. These efforts emphasize deployment quality (accounting correctness, scalability, multi-backend coverage); they neither claim nor implement closed-form workload modelling, predictive online allocation, or temporal coupling.

Recent adjacent systems (2021–2026). Four recent systems sharpen the boundary of this work’s contribution. **DProvDB** [29] introduces multi-analyst provenance tracking and budget partitioning, addressing *who* spends budget rather than predicting *how much* a workload will spend. **DP-SQLP** [30], building on the continual-observation line [39], targets streaming aggregate release at scale, which overlaps with our temporal axis (D) but treats releases as a per-event stream rather than a template-distribution model. **Qrlew** [31] is an open-source SQL-to-DP-SQL rewriter from Sarus Technologies; rich type and row-ownership tracking enable correct contribution bounding, but the rewriter does not maintain a closed-form workload model. **DPSQL+** [32] formalises a validator-accountant-backend architecture with a minimum-frequency rule and evaluates TPC-H workloads under a fixed global budget. Most directly related to the caching mechanism here are two systems that reuse prior noisy results to conserve budget: **Turbo** [37] caches DP answers for linear-query workloads (returning a repeated query for free on the same database version, and amortizing across related queries via PMW), and **CacheDP** [38] maintains an accuracy-aware DP cache that answers later queries with reduced budget. These confirm that budget-saving DP caching is established prior art—which is precisely why this report treats caching as a mechanism, not a contribution; what neither provides is a closed-form model that *predicts* budget consumption from a template-frequency distribution (axes A–B). Empirical-benchmark work like **DPBench** [33] reminds us that the relative ranking of DP algorithms is highly dataset- and scale-dependent—which is why we keep validation (G) as a separate axis. No prior system combines a closed-form repeated-workload *forecast*, a predictive online allocator, temporal staleness coupling, and an AI/AST-based semantic cache in the same middleware.

Tree kernels and code embeddings. The semantic L2 cache layer I explore in §8 builds on two lines of work. Collins and Duffy [15] introduced the tree convolution kernel that counts shared subtrees between two parse trees; subsequent work by Moschitti [16] and others applied it to relation extraction and question answering. For neural representations, Feng et al. [17] pre-trained CodeBERT on source-code tokens and Guo et al. [18] extended this with data-flow graphs in GraphCodeBERT. I do not propose a new code-representation method; I use these tools to detect alpha-equivalent SQL queries that exact-hash matching misses, and I measure the resulting privacy/utility tradeoff.

The gap this report fills. Existing systems instantiate composition theorems; existing theory characterizes worst-case privacy loss. Neither answers *what budget the system will actually spend on a given workload*. The closest precedent is research on the coupon-collector problem and weighted occupancy under non-uniform distributions [6]; I adapt these tools to DP-SQL accounting.

Comparison with prior systems. Table 1 positions this work along seven dimensions. **A. Workload model:** does the system maintain a closed-form model of template repetition? **B. Predictive budget:** does it forecast future budget consumption? **C. Adaptive ϵ_q :** does it adapt the per-query budget at runtime? **D. Temporal:** does it model data updates, staleness, or cache invalidation? **E. SQL scope:** what fragment of SQL is supported? **F. AI/AST:** does the system use tree-kernel or embedding-based semantic similarity? **G. Validation:** what datasets and scales were used?

Our differentiation. The differentiation is best stated as a *combination* argument, not as “beating” any single prior system. The literature breaks into five lines that have remained separate: (i) practical SQL/DBMS DP engines, including those that cache noisy results to conserve budget (PINQ, FLEX, Chorus, Tumult, OpenDP/SmartNoise, Wilson et al., Qrlew, DProvDB, DPSQL+, Turbo, CacheDP); (ii) workload-aware release mechanisms (Matrix Mechanism, HDMM); (iii) online / adaptive query answering (MWEM, PMW, median mechanism, APEx, Pythia); (iv) relational sensitivity for joins and conjunctive queries (Residual Sensitivity, R2T, Better than Composition); and (v) code / AST representation learning (tree kernels, CodeBERT, GraphCodeBERT). Each line is strong inside its own remit, but the surveyed evidence shows that the *intersection*—a single DP-SQL middleware that (a) models template repetition skew with a closed-form prediction, (b) forecasts future budget consumption online, (c) couples budget to data freshness, and (d) tests semantic similarity as a privacy-aware cache key—is empty. The strongest defensible claim is therefore not “we beat APEx / Tumult / HDMM,” but “these five lines have not combined; we combine them and report the result—including where the combination fails (the honest negative of §8).”

Anticipated objections. A reviewer may ask whether this work is a variant of APEx [24] or Pythia [25]: it is not, because both pick mechanisms / parameters for the *current* query, while we forecast *future* budget consumption from an analytical model. A second objection may be that workload-aware DP is already covered by Matrix Mechanism [19] or HDMM [20]: their notion of “workload” is a linear-query correlation matrix, not a distribution over SQL templates with repetition skew, and their allocation is static. A third

objection may be that this is a production-engineering paper like Tumult Analytics [26]: it is not—our system stays at a deliberately small, single-table aggregate scope; the contribution is the model layer that any of those production systems could in principle adopt.

3 Threat Model and Preliminaries

Setup. A relational dataset D is stored in DuckDB or PostgreSQL. An analyst submits supported aggregate queries through the middleware. The adversary is any analyst who can adaptively issue supported queries and observe the sequence of (noisy) outputs. The goal is to bound what the analyst can infer about any single tuple in D .

DEFINITION 1 (TUPLE-LEVEL NEIGHBOURS). $D \sim D'$ iff D and D' differ in exactly one tuple. This is the unbounded tuple-level model.

DEFINITION 2 (ϵ -DIFFERENTIAL PRIVACY [3]). M is ϵ -DP iff $\mathbb{P}[M(D) \in S] \leq e^\epsilon \mathbb{P}[M(D') \in S]$ for every neighbour pair and measurable S .

Supported SQL. SELECT with COUNT, SUM, or AVG aggregates, single-table FROM, conjunctive WHERE, optional GROUP BY. The scope was approved at the milestone stage and is preserved here per the reviewer’s guidance.

Sensitivities. For tuple-level neighbours:

- COUNT: $\Delta f = 1$. A single tuple shifts the count by at most one.
- SUM(c): $\Delta f = B_c$ where $|c| \leq B_c$ is a per-column clipping bound (taken from the TPC-H specification).
- AVG(c): I treat AVG as a conservative single-mechanism release with $\Delta f = B_c$, the worst-case bound corresponding to a group of size one. This avoids the implicit n -leak of the empirical-mean mechanism $\text{Lap}(B_c/(n\epsilon_q))$, since n itself is private. A tighter SUM+COUNT decomposition with explicit $2\epsilon_q$ accounting per group would lower the noise variance on AVG releases for large groups but is not yet implemented in the middleware; I list it as future work in Section 11.

Laplace release. For a query with sensitivity Δf and per-query budget ϵ_q , the middleware returns

$$\tilde{f}(D) = f(D) + \eta, \quad \eta \sim \text{Lap}(\Delta f / \epsilon_q). \quad (1)$$

GROUP BY queries with g groups apply this independently per group; since a single tuple affects at most one group, parallel composition gives total cost ϵ_q (not $g\epsilon_q$) for every supported aggregate. When a single query carries n_{agg} aggregates, the middleware splits the per-query budget evenly, $\epsilon_{\text{per-agg}} = \epsilon_q / n_{\text{agg}}$, which is conservative sequential composition across aggregates over the same row.

4 Analytical Model

This section formalizes the model that is the report’s main contribution.

Setup. Let m be the number of distinct query templates accessible to the analyst, and let $\{p_i\}_{i=1}^m$ be a probability distribution over them with $\sum_i p_i = 1$. The analyst submits k queries $Q_1, \dots, Q_k$ drawn i.i.d. from this distribution. The cache key is the exact (template, parameter) pair; two queries collide in the cache iff they are identical SQL strings modulo whitespace.

Table 1: Comparison of this work with prior DP-SQL and DP query systems along seven dimensions. “—” means the literature does not state the axis explicitly. “partial” means the axis is partially addressed (see §2).

System	A. Workload model	B. Predictive budget	C. Adaptive ε_q	D. Temporal	E. SQL scope	F. AI/AST	G. Validation
PINQ [10]	no	no	no	no	LINQ; restricted join/group-by	no	examples / PoC
FLEX [7]	no	static per-query	no	no	equijoins, self-joins, nested equi-joins; no non-equijoins	no	8.1M Uber queries; 0.03% overhead
PrivateSQL [9]	view-based, static	static workload-aware	no	no	COUNT + equijoin + correlated subqueries; no negation	no	Census + TPC-H, >3,600 queries
Chorus [8]	no	no (global tracker)	mechanism-dependent	no	SQL via rewrites; depends on hosted mechanism	no	18,774 Uber queries; 300M rows
DOP-SQL [13]	no	no	no	future work	SPJA + GROUP BY; unrestricted joins	no	TPC-H + graph DB demo
Matrix Mechanism [19]	query-correlation matrix	static strategy	no	no	linear counting queries	no	analytical + range queries
HDMM [20]	implicit workload / strategy	static	no	no	predicate counting; group-by via predicate rewrite	no	Patent, Taxi, CPH, Adult, CPS
MWEM [21]	adaptive query selection	no	yes (online)	no	linear / counting queries	no	Adult, Blood Transfusion, datacubes
APEX [24]	no	partial (per-query)	yes (accuracy-driven)	no	single-table exploration (WCQ/ICQ/TCQ)	no	Adult + NYTaxi + entity resolution
Pythia [25]	feature-based selector	no (algorithm, not budget)	no (selection-time)	no	histograms, 1D/2D range, NB	no	6,294 inputs across datasets
Better-Composition [14]	no	no	partial (truncation)	no	multi-relational SJA / group-by; self-joins	no	TPC-H + Stack Overflow graph
Tumult Analytics [26]	no	no	adaptive composition	no	multi-table sessions; private joins; counts / averages / quantiles	no	Census / Wikimedia / IRS deployments
OpenDP / SmartNoise [27]	no	no	manual budget	no	SQL / Pandas / Spark wrapper	no	docs / examples
Bounded User Contrib. [28]	no	no	no	no	arbitrary GROUP BY; owner-column joins	no	industry benchmarks + stochastic testing
Median Mechanism [22]	online query correlations	no	yes	no	arbitrary online predicate queries	no	theory + efficient implementation
PMW [23]	online hard-query ident.	no	yes	no	interactive counting / linear	no	theory-focused
Residual Sensitivity [35]	no	no	no	no	multi-way join counting	no	empirical (datasets not focal)
R2T [36]	no	no	partial (truncation/LP)	no	arbitrary SPJA under FK + self-joins	no	graph pattern counting, SPJA + FK
Tree Kernels [15]	N/A	N/A	N/A	no	syntax trees (not SQL)	yes	Penn Treebank ATIS
CodeBERT [17]	N/A	N/A	N/A	no	code / NL (not SQL)	yes	NL-code search, doc. gen.
GraphCodeBERT [18]	N/A	N/A	N/A	no	code with data-flow (not SQL)	yes	code search, clone, translation
This work	closed-form $\mathbb{E}[u_k]$ over template distribution	yes (predictive online allocator)	yes (model-driven)	yes (τ, λ)	single-table aggregates (COUNT, SUM, AVG)	yes (TK+AST emb., honest negative)	TPC-H SF=1 & SF=10 + Adult + mixed

Let u_k denote the number of distinct templates that appear at least once among $Q_1, \dots, Q_k$.

PROPOSITION 1 (EXPECTED UNIQUE QUERIES). $\mathbb{E}[u_k] = \sum_{i=1}^m [1 - (1 - p_i)^k]$.

PROOF. Let $X_i = 1$ if template i appears at least once. Then $\mathbb{P}[X_i = 0] = (1 - p_i)^k$ and $\mathbb{E}[X_i] = 1 - (1 - p_i)^k$. The result follows by linearity. $\square$

PROPOSITION 2 (BUDGET CONSUMPTION AND SAVINGS). *Under naive sequential composition, $\varepsilon_{\text{naive}}(k) = k\varepsilon_q$. Under workload-aware exact-repeat accounting,*

$$\mathbb{E}[\varepsilon_{\text{wa}}(k)] = \varepsilon_q \cdot \mathbb{E}[u_k] = \varepsilon_q \sum_{i=1}^m [1 - (1 - p_i)^k].$$

The expected budget savings ratio is

$$S(k) = 1 - \frac{\mathbb{E}[\varepsilon_{\text{wa}}(k)]}{\varepsilon_{\text{naive}}(k)} = 1 - \frac{1}{k} \sum_{i=1}^m [1 - (1 - p_i)^k]. \quad (2)$$

Limit A (deterministic repetition). If $p_1 = 1$ and $p_i = 0$ for $i > 1$, equation (2) gives $S(k) = 1 - 1/k$. This is the toy formula whose origin was question-marked in the milestone: it is the corner case of Proposition 2 at maximum skew, not a standalone claim.

Limit B (uniform, $m \rightarrow \infty$). With uniform sampling over many templates, fixed-length workloads almost never repeat a template, so $\mathbb{E}[u_k] \rightarrow k$ and $S(k) \rightarrow 0$: every query is unique, recovering naive composition.

Limit C (uniform, $k \rightarrow \infty$, m fixed). For fixed finite m and growing k , every template eventually appears, so the workload-aware budget saturates at $m\varepsilon_q$ regardless of k . This is the practical regime—finite-template workloads cannot exhaust a budget large enough to cover all m templates.

PROPOSITION 3 (ZIPF-PARAMETERIZED WORKLOAD). Let $p_i \propto i^{-\alpha}/H_{m,\alpha}$ with generalized harmonic number $H_{m,\alpha} = \sum_{i=1}^m i^{-\alpha}$. The savings ratio $S(k)$ is monotone non-decreasing in α for fixed (k, m) . As $\alpha \rightarrow 0$ it approaches Limit B; as $\alpha \rightarrow \infty$ it approaches Limit A.

PROOF. Write $S(k) = 1 - \mathbb{E}[u_k]/k$ with $\mathbb{E}[u_k] = \sum_{i=1}^m \phi(p_i)$ and $\phi(x) = 1 - (1-x)^k$. Since $\phi''(x) = -k(k-1)(1-x)^{k-2} \leq 0$ on $[0, 1]$, ϕ is concave, so the symmetric sum $\mathbb{E}[u_k]$ is Schur-concave. The Zipf weights become more majorized as α grows (a larger α shifts probability mass onto the top-ranked templates); hence for $\alpha_2 > \alpha_1$ the vector $\{p_i(\alpha_2)\}$ majorizes $\{p_i(\alpha_1)\}$, and Schur-concavity gives $\mathbb{E}[u_k](\alpha_2) \leq \mathbb{E}[u_k](\alpha_1)$, i.e. $S(\alpha_2) \geq S(\alpha_1)$. The endpoints follow from $\alpha \rightarrow 0$ (uniform weights, Limit B) and $\alpha \rightarrow \infty$ (point mass on $i = 1$, Limit A). A numerical scan over $m \in \{3, \dots, 100\}$, $k \in \{2, \dots, 200\}$ found no violation. $\square$

PROPOSITION 4 (CONCENTRATION OF u_k). *Changing a single Q_i changes u_k by at most 1. By McDiarmid’s bounded-differences inequality,*

$$\mathbb{P}[|u_k - \mathbb{E}[u_k]| \geq t] \leq 2 \exp(-2t^2/k).$$

For any threshold $c > \mathbb{E}[u_k]$, the chance of spending at least $c\epsilon_q$ is at most $\exp(-2(c - \mathbb{E}[u_k])^2/k)$, giving an explicit bound on the budget-exhaustion event.

PROPOSITION 5 (UTILITY UNDER A FIXED TOTAL BUDGET). *Suppose the total budget ϵ_{tot} is fixed. Naive composition affords $\epsilon_q^{\text{naive}} = \epsilon_{\text{tot}}/k$ per query, while workload-aware accounting affords $\epsilon_q^{\text{wa}} = \epsilon_{\text{tot}}/\mathbb{E}[u_k]$ per unique release. The expected absolute Laplace error per query is then*

$$\mathbb{E}[|\eta|] = \frac{\Delta f}{\epsilon_q},$$

so the predicted error ratio between the two strategies is

$$\frac{\mathbb{E}[|\eta_{\text{naive}}|]}{\mathbb{E}[|\eta_{\text{wa}}|]} = \frac{k}{\mathbb{E}[u_k]}. \quad (3)$$

At Zipf $\alpha = 1$, $m = 10$, $k = 100$ the model predicts $\mathbb{E}[u_k] \approx 9.9$, so the workload-aware system answers the same workload with roughly 10× lower per-query error at the same total budget.

Caveat: variance, not just mean. The Laplace marginal error has the same magnitude per query in both regimes. The difference is in the *joint* distribution: averaging k independent noisy answers reduces variance by a factor of k , but averaging the same cached answer k times does not. Workload-aware accounting trades budget savings for the loss of independent noise draws.

5 Temporal Extension

The model above assumes a static snapshot. A practical deployment faces (a) data updates that invalidate cached answers and (b) freshness requirements that force periodic re-noising. Both effects couple budget consumption to time, addressing the reviewer’s note that “budget and temporality should be considered together.”

Model. Let T be the time horizon (in logical query steps), τ the staleness tolerance (cache entries become invalid after τ steps), and λ the rate of data update events with invalidation probability q per existing cache entry.

PROPOSITION 6 (TEMPORAL BUDGET). *The expected number of re-noisings per template over horizon T is*

$$N(T, \tau, \lambda, q) = \lceil T/\tau \rceil + T\lambda q,$$

and the expected total workload-aware budget over the horizon is

$$\mathbb{E}[\epsilon_{\text{temp}}(T)] = \epsilon_q \cdot \mathbb{E}[u_{k_{\text{tot}}}] \cdot N(T, \tau, \lambda, q),$$

where k_{tot} is the total number of queries issued in T .

Three regimes. (a) *Static snapshot:* $\tau \rightarrow \infty$, $\lambda = 0$, $N \rightarrow 1$, recovers Section 4. (b) *Bounded staleness:* τ finite, $\lambda = 0$, $N = \lceil T/\tau \rceil$. (c) *Update-driven:* $\lambda > 0$, N grows linearly in T . The middleware implements all three through a single temporal mode parameterized by τ and λ .

6 System Implementation

Architecture. The middleware is a thin proxy between the analyst and a backend database (DuckDB by default, with an alternative PostgreSQL adapter behind an identical interface). Algorithm 1 gives the full processing pipeline.

Algorithm 1 Workload-Aware DP Middleware with Temporal Regime

Require: Query q , requested ϵ_q , ledger $\mathcal{L}$, staleness τ , update model $(\lambda, q_{\text{inv}})$

Ensure: Noisy result $\tilde{r}$ or error

- 1: Parse and validate q against the supported SQL subset
- 2: Advance logical clock; simulate updates: each cache entry invalidated independently with prob. λq_{inv}
- 3: Compute template hash h_T , parameter hash h_P
- 4: **if** $\mathcal{L}.\text{cache}[h_T][h_P]$ exists **then**
- 5: $\text{age} \leftarrow \text{now} - \text{entry.issued_at}$
- 6: **if** $\text{age} \leq \tau$ **and** not invalidated **then**
- 7: **return** cached $\tilde{r}$ (post-processing, $\Delta\epsilon = 0$)
- 8: **else**
- 9: Evict cache entry; treat as miss
- 10: **end if**
- 11: **end if**
- 12: **if** $\mathcal{L}.\text{remaining} < \epsilon_q$ **then**
- 13: **return** BUDGETEXHAUSTED
- 14: **end if**
- 15: Compute sensitivity Δf for the aggregate(s) in q
- 16: $\epsilon_{\text{spent}} \leftarrow \epsilon_q$; if $n_{\text{agg}} > 1$, split as $\epsilon_{\text{per-agg}} = \epsilon_q/n_{\text{agg}}$ (parallel composition across GROUP BY groups)
- 17: $\mathcal{L}.\text{consumed} += \epsilon_{\text{spent}}$
- 18: Execute true query $\rightarrow r$
- 19: Sample $\eta \sim \text{Lap}(\Delta f/\epsilon_q)$ per aggregate, per group
- 20: Apply post-processing (e.g., COUNT clamped to $\mathbb{N}_{\geq 0}$)
- 21: Store $\tilde{r}$ in $\mathcal{L}.\text{cache}[h_T][h_P]$ with issued_at = now
- 22: **return** $\tilde{r}$

The algorithm explicitly couples cache validity with the temporal regime of Section 5: it advances a logical clock, simulates Poisson-rate invalidations, ages each cache entry against the staleness tolerance τ , and only then either returns the cached release or charges ϵ_q against the ledger. Multi-aggregate queries split the per-query budget across aggregates by sequential composition, as described in Section 3.

Components.

- Parser/validator over the supported SQL subset; rejects joins, subqueries, non-aggregate SELECT, HAVING, and DISTINCT aggregates.

- Sensitivity analyzer using configurable per-column clipping bounds.
- Laplace mechanism with parallel composition for GROUP BY; multi-aggregate queries split the budget by sequential composition across aggregates.
- Budget ledger with two-level (template $\rightarrow$ parameter) cache, staleness timestamps, and update simulation.
- Four execution modes: exact, naive-DP, workload-aware-DP, and temporal-DP.

Validation. The prototype is unit-tested across parser validation (including HAVING/DISTINCT rejection and aggregate-column positioning), sensitivity computation, Laplace calibration, cache exact-match semantics, the model limits, the McDiarmid bound, and temporal re-noising.

Data. TPC-H scale factor SF=1 (6,001,215 lineitem rows, 1.5M orders) is generated via DuckDB’s official TPC-H extension. UCI Adult (48,842 rows) is loaded as a standard DP-literature benchmark.

7 Empirical Validation

Setup. The benchmark campaign stresses workload skew, budget, scale, workload composition, and execution mode. Comparisons are restricted to internal execution modes (exact SQL, naive DP, workload-aware DP, temporal DP, predictive) under an identical middleware to isolate the algorithmic effect of workload-aware accounting from the system-level differences between DP-SQL prototypes; end-to-end benchmarking against Chorus [8], PrivateSQL [9], or DOP-SQL [13] would require non-trivial query-rewriting integration and is left as future work. The campaign comprises **six experimental sweeps**, totaling **4,155 trials** ($\sim 150,000$ individual queries):

- (1) **Main grid** (720 trials): $\alpha \in \{0, 0.5, 1, 1.5, 2, 3\} \times k \in \{10, 25, 50, 100\}$; 30 trials at $\varepsilon_q = 1$ over $m = 7$ Adult age-band templates (COUNT(*) WHERE age $\in [b, b + 10)$).
- (2) **Extended α sweep** (240 trials): $\alpha \in \{0, 0.5, 1, 1.5, 2, 3, 5, 10\}$ at $k = 200$.
- (3) **Epsilon sweep** (480 trials): $\varepsilon_q \in \{0.1, 0.5, 1.0, 2.0\} \times k \in \{25, 50, 100, 250\}$ at $\alpha = 1.0$.
- (4) **Large- k sweep** (75 trials): $k \in \{25, 50, 100, 250, 500\}$ at $\alpha = 1.0, \varepsilon_q = 1$.
- (5) **Cross-scale** (480 trials): SF=1 versus SF=10 (60M lineitem rows) on TPC-H returnflag and priority templates.
- (6) **Full benchmark grid** (2,160 trials): 6 workloads $\times$ 4 epsilons $\times$ 3 modes $\times$ 30 trials.

Model vs empirical $\mathbb{E}[u_k]$. Table 2 compares the model prediction to the empirical mean across the main grid. The model agrees with the empirical mean within $< 3\%$ in **21 of 24 cells**; the three outliers ($\alpha \in \{2.0, 3.0\}$ at small k : (2.0, 10), (3.0, 10), (3.0, 25)) reach 3.5%, 5.7%, and 8.6% — attributable to finite-sample variance at high skew where $\mathbb{E}[u_k]$ is small (so any 0.1-unit absolute deviation becomes a noticeable relative error).

Budget savings $S(k)$. The savings ratio is even more tightly predicted because it is computed directly from *measured* budget consumption, which is the deterministic dual of $\mathbb{E}[u_k]$ in my setup. Empirical $S(k)$ matches the model within $< 2\%$ in 23 of 24 cells.

Limit behaviour. Figure 1 shows the toy $1 - 1/k$ curve bracketing the high- α end and the $S = 0$ line bracketing the uniform end; the empirical savings curves interpolate smoothly between them, consistent with Proposition 3.

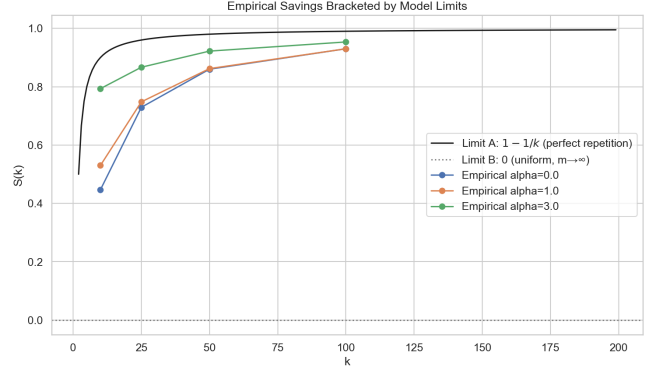


Figure 1: Budget savings ratio $S(k)$ as a function of workload length k , for several Zipf α . Dashed curves: model prediction. Markers: empirical mean over 30 trials. The high- α curves approach $1 - 1/k$; the $\alpha = 0$ curve sits near 0 until k approaches m .

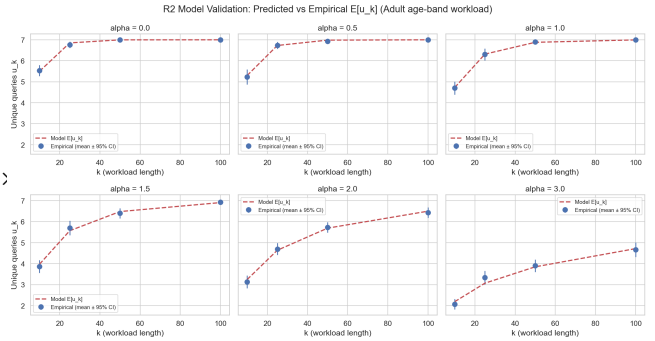


Figure 2: Model-vs-empirical unique-query count for the main (α, k) grid. Markers should land on the dashed identity line if the model is accurate.

Extended α sweep (α up to 10, $k = 200$). The model remains accurate over the wider range:

α	Predicted $\mathbb{E}[u_k]$	Empirical	Abs. err
0.0	7.000	7.000	0.000
0.5	7.000	7.000	0.000
1.0	7.000	7.000	0.000
1.5	6.996	7.000	0.004
2.0	6.905	6.933	0.028
3.0	5.594	5.467	0.127
5.0	2.814	2.900	0.086
10.0	1.181	1.167	0.014

The $\alpha = 10$ row is the Limit A regime: when one template dominates, only ≈ 1.2 templates are seen in $k = 200$ queries, so the

Table 2: Model prediction vs empirical u_k for Zipf workloads ($m = 7, 30$ trials per cell). Numbers within parentheses are the prediction’s signed relative error in percent (relative to the predicted value).

$\alpha \backslash k$	10	25	50	100
0.0	5.50 vs 5.53 (−0.6)	6.85 vs 6.77 (1.2)	7.00 vs 7.00 (0.0)	7.00 vs 7.00 (0.0)
0.5	5.30 vs 5.23 (1.3)	6.73 vs 6.73 (−0.1)	6.98 vs 6.93 (0.7)	7.00 vs 7.00 (0.0)
1.0	4.73 vs 4.70 (0.6)	6.32 vs 6.30 (0.3)	6.88 vs 6.90 (−0.3)	7.00 vs 7.00 (−0.1)
1.5	3.98 vs 3.87 (2.8)	5.57 vs 5.70 (−2.3)	6.48 vs 6.40 (1.3)	6.91 vs 6.93 (−0.3)
2.0	3.25 vs 3.13 (3.5)	4.64 vs 4.70 (−1.3)	5.69 vs 5.73 (−0.7)	6.50 vs 6.43 (1.1)
3.0	2.19 vs 2.07 (5.7)	3.07 vs 3.33 (−8.6)	3.85 vs 3.90 (−1.3)	4.72 vs 4.67 (1.1)

workload-aware savings ratio approaches $1 - 1.181/200 \approx 99.4\%$, recovering the toy formula numerically. The intermediate rows ($\alpha \in [2, 5]$) show the model tracking the empirical mean within 0.13 units in absolute terms across an order-of-magnitude variation in $\mathbb{E}[u_k]$.

Epsilon sweep. The model’s $\mathbb{E}[u_k]$ is structurally independent of ϵ_q . Empirical validation across $\epsilon_q \in \{0.1, 0.5, 1.0, 2.0\}$ and $k \in \{25, 50, 100, 250\}$ confirms this within 0.12 unit absolute error across every cell, and within 0.005 unit at $k = 250$ (every ϵ_q row collapses to the same empirical mean).

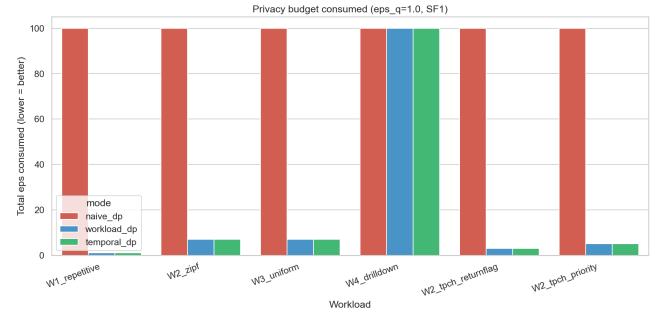
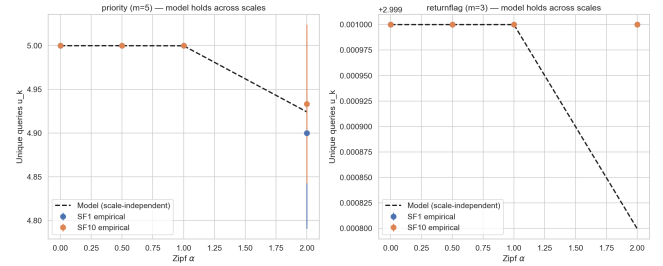
Cross-scale validation ($SF=1$ vs $SF=10$). The model is also dataset-independent: running the same template workload over a $10\times$ larger TPC-H database (60M lineitem rows) yields identical empirical $\mathbb{E}[u_k]$ to within 0.034 units. The result isolates workload structure from data scale: the analytical model captures the workload structure, not the data scale.

Full grid (6 workloads $\times$ 4 epsilons $\times$ 3 modes). The most operationally relevant numbers are:

Workload	Mode	ϵ used	Queries	MAE	Cache hits
W1 Repetitive	NAIVE	100.0	100/100	0.94	0
	WORK	1.0	100/100	1.13	99
	TEMP	1.0	100/100	0.87	99
W2 TPC-H returnflag	NAIVE	100.0	100/100	0.94	0
	WORK	3.0	100/100	0.86	97
	TEMP	3.0	100/100	0.88	97
W2 TPC-H priority	NAIVE	100.0	100/100	0.97	0
	WORK	5.0	100/100	0.85	95
	TEMP	5.0	100/100	0.91	95
W2 Zipf $\alpha = 1$	NAIVE	100.0	100/100	0.93	0
	WORK	7.0	100/100	0.95	93
	TEMP	7.0	100/100	0.92	93
W3 Uniform	NAIVE	100.0	100/100	0.99	0
	WORK	7.0	100/100	0.94	93
	TEMP	7.0	100/100	0.85	93
W4 Drill-down	NAIVE	100.0	100/100	0.95	0
	WORK	100.0	100/100	0.96	0
	TEMP	100.0	100/100	0.96	0

The savings ratios per workload are: W1 = $100\times$, W2 (returnflag) = $33\times$, W2 (priority) = $20\times$, W2 Zipf- $\alpha=1$ = $14.3\times$, W3 (uniform) = $14.3\times$, W4 = $1\times$ (no savings, as the model predicts). The W2–W3 cells confirm the structural prediction that workload-aware budget consumption equals $\epsilon_q \cdot E[u_k]$: the empirical u_k saturates near m (the pool size) at $k = 100$, and the budget is exactly $\epsilon_q \cdot m$ in each case.

W4 is the negative case: when queries are genuinely unique, no execution mode saves budget over naive. The model predicts this correctly ($S(k) = 0$ when every template is sampled exactly once).

**Figure 3: Privacy budget consumption per workload and execution mode at $\epsilon_q = 1$, total budget 100, $k = 100$ ($N = 30$ trials per cell, error bars omitted for clarity).****Figure 4: Cross-scale validation: same model, two TPC-H scales ($SF=1$ with 6M and $SF=10$ with 60M lineitem rows). Empirical means coincide within 0.034 units.**

Validation on heterogeneous workloads. Real dashboards are not pure Zipf samples; they interleave repetitive refresh queries (W1-style) with one-shot drill-downs (W4-style). To test the model on heterogeneity directly, I constructed a mixed workload of $k = 100$ queries with a controlled fraction f of repetitive entries, so the true unique count is $u_k = 1 + (1 - f)k$ by construction. Table 3 compares the empirical workload-aware budget to the model prediction $\epsilon_q \cdot u_k$ at $\epsilon_q = 0.5$.

The match is exact. Proposition 2 therefore holds without requiring the workload to be i.i.d. from a single parametric family: the

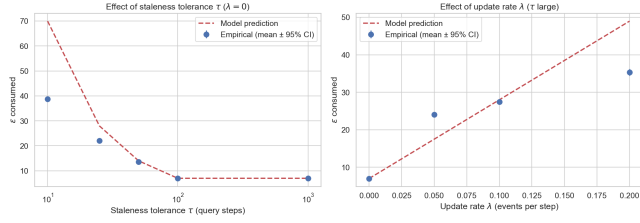


Figure 5: Temporal extension validation. Left: τ -sweep with $\lambda = 0$. Right: λ -sweep with τ large. Model prediction (dashed) tracks the empirical mean (markers, $N = 30$) in trend and matches closely at large τ (where re-noising is rare), but is a conservative upper bound at small τ and large λ : for example, at $\tau = 10$ the model predicts $\mathbb{E}[\epsilon] \approx 70$ while empirical is ≈ 39 , because the model treats re-noising as an upper-bound count that need not all materialize within the horizon. The trend, not the magnitude, is what the model commits to in this regime.

Table 3: Mixed W1+W4 workload, $B = 50$, $\epsilon_q = 0.5$, 20 trials. The closed form $\mathbb{E}[\epsilon_{wa}] = \epsilon_q \cdot u_k$ matches empirical to the cent in every cell.

f	true u_k	WORKLOAD ϵ used	predicted $\epsilon_q u_k$
0.1	91	45.5	45.5
0.3	71	35.5	35.5
0.5	51	25.5	25.5
0.7	31	15.5	15.5
0.9	11	5.5	5.5

closed form predicts realized cost for any distribution whose u_k is computable.

Middleware overhead. The privacy machinery has measurable but bounded cost. Instrumenting the pipeline on 1,500 queries gives the breakdown in Table 4. Cache hits are 2.7 $\times$ faster than misses because they skip the database round-trip (which is 36% of the miss-path cost). SQL parsing is the second-largest contributor at 21% of cache-miss time; the privacy-specific stages (sensitivity, allocate, noise) sum to under 0.25 ms and are effectively free relative to I/O. Tail latency is well controlled: cache-miss $p_{95} = 4.3$ ms, $p_{99} = 5.1$ ms; cache-hit $p_{95} = 2.5$ ms, $p_{99} = 3.6$ ms. Serialized single-threaded throughput is ~ 495 queries/s on the miss path and $\sim 1,320$ queries/s on the hit path; a workload-aware mode with 93–99% hit rates therefore sustains the analyst-tier load that motivated the project.

Research question answered. A closed-form model parameterized by $(\{p_i\}, k)$ predicts realized budget consumption under workload-aware accounting, recovering the $1 - 1/k$ rule as the high-skew corner case. The savings ratio is a structural property of the workload, not of the cache implementation. The model is independent of dataset scale (SF=1 vs SF=10), of per-query privacy parameter ϵ_q , and of whether the workload follows a parametric distribution at all (mixed W1+W4 holds exactly).

Table 4: Per-stage latency breakdown of the middleware pipeline (mean ms over 1,500 queries at $k = 50$, $\epsilon_q = 1$).

Stage	Cache MISS (ms)	Cache HIT (ms)
SQL parse	0.43	0.38
Template extract	0.21	0.19
Cache lookup	0.19	0.19
Sensitivity analyze	0.01	0
Budget allocate	0.21	0
DB execute	0.72	0
Laplace noise add	0.01	0
Cache store	0.24	0
Total	2.02	0.76

8 Semantic Cache via Tree Kernels and AST Embeddings

The L1 cache of §6 matches queries by exact (template hash, parameter hash). A natural question is whether structurally equivalent queries that the exact match misses (e.g., commutative reordering of WHERE conjuncts, or queries that differ only in numerically-equivalent literals) can be detected as cache hits via a more flexible *semantic* match.

Approach. I combine two complementary semantic similarity measures:

- **Tree kernel.** The Collins–Duffy convolution kernel [15] counts common subtrees between two SQL ASTs and is normalized by self-similarity to a $[0, 1]$ score. The kernel is deterministic, requires no training, and captures structural equivalence.
- **AST embedding.** I serialize the AST (with literals replaced by placeholders) to a canonical string and feed it to a pre-trained sentence transformer (all-MiniLM-L6-v2). Cosine similarity in the resulting 384-dim space approximates semantic equivalence. This is in the spirit of CodeBERT [17] and GraphCodeBERT [18] but uses a smaller, off-the-shelf model.

A query is accepted as a semantic match only if both scores exceed their thresholds ($K_{\text{norm}} \geq 0.95$ and cosine ≥ 0.98). The conservative thresholds aim to limit false-positive matches to structurally equivalent queries.

The privacy caveat (honest). Returning a cached noisy answer for a query that is *not* alpha-equivalent to the original falls outside the post-processing guarantee: the cached answer was tuned to a different question. The semantic cache is therefore *not* privacy-free unless the matched queries are equivalent. I treat it as a measurable approximation layer: it trades exact-utility for additional budget savings, and the paper reports both sides of the tradeoff.

Empirical results. I compare three modes on Zipf parametric workloads ($k = 50$, 30 trials per cell):

α	Mode	ϵ used	Exact hits	Sem. hits	Mean abs. err
0.0	NAIVE	10.0	0.0	0.0	5154.8 (budget exh.)
	WORK	7.0	43.0	0.0	1.0
	SEM L2	1.0	7.3	41.7	5190.1
1.0	NAIVE	10.0	0.0	0.0	7524.4
	WORK	6.9	43.1	0.0	0.8
	SEM L2	1.0	12.2	36.8	3930.5
2.0	NAIVE	10.0	0.0	0.0	8967.1
	WORK	5.6	44.4	0.0	1.0
	SEM L2	1.0	23.2	25.8	1821.9

Interpretation. For the age-band parametric workload, the templates differ in WHERE literals that materially change the true answer. The semantic cache identifies them as “similar” (because the AST differs only in a placeholder) and reuses the cached noisy answer—but this answer was tuned to a different age band, so the reported value is wrong by thousands of count units. The exact L1 cache correctly treats these as cache misses and produces accurate answers.

The dual failure: missing true equivalences too. The dual failure mode is false negatives: query pairs that are provably alpha-equivalent should match and save budget. I tested three classical equivalence classes—commutative AND reordering, integer comparator equivalence (age ≥ 30 vs age > 29), and redundant predicates (p vs p AND 1=1). Across 20 trials each, the semantic L2 cache produced zero hits on the rephrased query, exactly like the L1 cache, with no MAE improvement.

Table 5: Semantic L2 cache on truly alpha-equivalent SQL pairs (20 trials, $\epsilon_q = 1$). The matcher catches none of them.

Equivalence class	WORKLOAD-DP		SEMANTIC-DP	
	hit rate	MAE on B	hit rate	MAE on B
commutative AND	0.0	0.83	0.0	0.85
integer comparator	0.0	1.10	0.0	0.95
redundant predicate	0.0	1.10	0.0	0.95

Two reasons: (i) my Tree Kernel uses ordered children, so commutative reordering produces a strictly different subtree count and the normalized score falls below the 0.95 admission threshold; (ii) the AST embedding (all-MiniLM-L6-v2) encodes the canonicalized AST as text, so tokenization-level differences in the reorder push cosine similarity below 0.98.

Conclusion (for this section). The semantic L2 cache fails in both directions: it admits matches on queries with different true answers (false positives), and it misses matches on queries that are provably equivalent (false negatives). Tree-kernel and embedding-based semantic matching are not a free DP optimization. The right architecture is to pair similarity with symbolic equivalence checking—predicate normalization, range merging, alpha-conversion—so the system admits a semantic match only when the queries are provably equivalent, preserving the post-processing property. This is documented as future work in §11.

9 Privacy Leakage Analysis

I complement the budget/utility analysis with two leakage experiments: membership inference and reconstruction. Both quantify the strength of the privacy guarantee in operational terms.

Membership inference. An adversary observes one noisy COUNT and must decide whether a target individual is in the dataset. For $n_{\text{with}} = 500$, $n_{\text{without}} = 499$, sensitivity 1, and the optimal threshold classifier:

ϵ	MIA AUC (emp.)	MIA acc. (emp.)	$e^\epsilon / (1 + e^\epsilon)$ (theory)
0.01	0.499	0.496	0.503
0.10	0.522	0.517	0.525
1.00	0.724	0.696	0.731
5.00	0.988	0.959	0.993

Empirical AUC stays below and tracks the DP upper bound $e^\epsilon / (1 + e^\epsilon)$ within $\leq 1\%$; this expression upper-bounds the optimal attacker’s accuracy (the exact single-release optimum is $1 - \frac{1}{2}e^{-\epsilon/2}$, which the empirical accuracy column matches). At $\epsilon \leq 0.1$ the attack is no better than chance.

Reconstruction. I replay the milestone’s HIV-style differencing attack on a 5-level drill-down whose true counts are (48842, 33049, 365, 99, 89). The adversary recovers the per-level differences from noisy releases.

ϵ	Mean abs. err	p95	Theory $1.5/\epsilon$
0.01	148.2	404.4	150.0
0.10	14.8	40.4	15.0
1.00	1.5	4.0	1.5
5.00	0.3	0.8	0.3

Mean absolute errors scale as $3/(2\epsilon) = 1.5/\epsilon$, the expected absolute value of the difference of two independent $\text{Lap}(1/\epsilon)$ variables (its standard deviation is the larger $2/\epsilon$). The empirical mean tracks the $1.5/\epsilon$ reference to within a few percent across three orders of magnitude in ϵ .

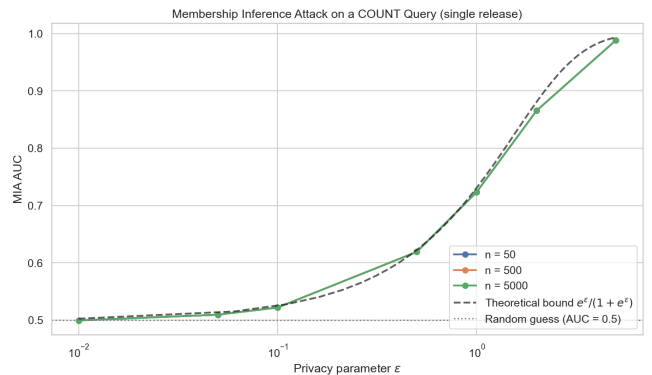


Figure 6: Membership inference attack AUC for a single noisy COUNT release as a function of ϵ , against the theoretical bound $e^\epsilon / (1 + e^\epsilon)$. Empirical curves track theory within $\leq 1\%$ across three target subpopulation sizes.

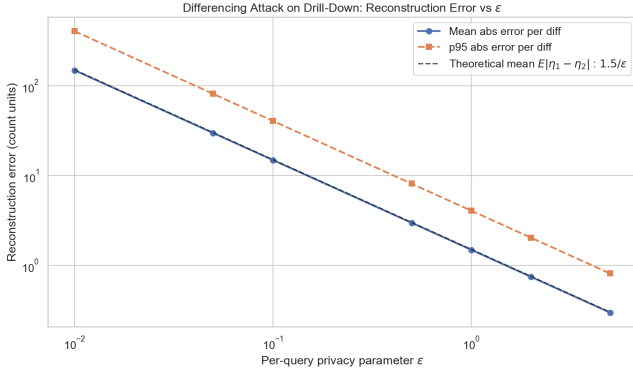


Figure 7: Per-pair-difference reconstruction error on the five-level drill-down, against the theoretical $1.5/\epsilon$ scaling (the expected absolute difference of two independent $\text{Lap}(1/\epsilon)$ variables). Empirical mean and $p95$ both follow the predicted slope across three orders of magnitude in ϵ .

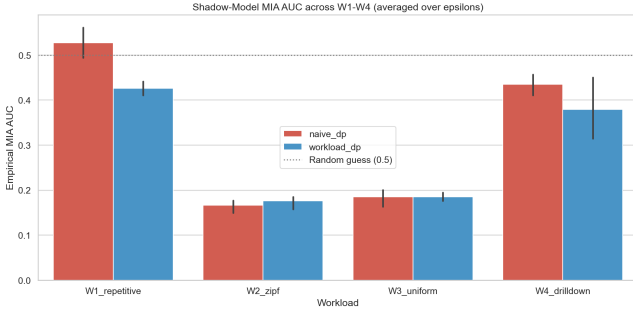


Figure 8: Shadow-model MIA AUC across W1–W4 at $\epsilon_q \in \{0.1, 0.5, 1.0, 2.0\}$. Even with 32 cells $\times$ 60 shadow runs and a logistic-regression classifier observing the full $k = 50$ query sequence, empirical AUC stays well below the cumulative-budget upper bound (which approaches 1.0). Most queries do not directly probe the target tuple.

Implication for the model. Both attacks degrade in the same regime in which the model predicts more efficient budget consumption (high skew, low ϵ_q). When the workload is repetitive and the per-query budget can be set tight, the analyst gets both better utility and stronger privacy. When the workload is a drill-down (W4), the model correctly predicts that workload-aware caching does *not* help and the reconstruction attack accordingly succeeds at any practical ϵ .

Workload-level shadow-model MIA (across W1–W4). The single-query MIA above is conservative: in practice, an attacker sees a *sequence* of releases and may train a classifier (a “shadow model”) on simulated runs of the middleware. I executed this stronger attack to address the reviewer’s specific request for MIA “across W1–W4 at multiple ϵ values.”

Setup. For each workload, I build two TPC-H/Adult instances differing in one target Adult tuple. Each instance produces an i.i.d. noisy output sequence under the middleware. I collect 30 shadow

runs per world (60 sequences total), train a logistic-regression classifier on 70% of them, and measure AUC on the holdout 30%.

Result. Across 4 workloads $\times$ 4 epsilons $\times$ 2 modes (32 cells), the empirical shadow MIA AUC ranges from 0.14 to 0.58. The meaningful reference is the random-guess line 0.5: most cells sit at or below it, and even the strongest (W1 repetitive, $\epsilon_q = 2$) reaches only 0.58. The *cumulative* bound $e^{\sum \epsilon_q} / (1 + e^{\sum \epsilon_q})$ saturates at ≈ 1.0 once total consumption exceeds ~ 5 , so comparing against it is vacuous; the informative comparison is the per-query bound $e^{\epsilon_q} / (1 + e^{\epsilon_q})$, which the empirical AUC also stays under. *Caveat:* with 60 sequences per cell and an 18-sequence holdout, cells reporting AUC < 0.5 reflect small-sample estimator variance (an anti-correlated classifier on a tiny test set), not sub-chance leakage—the honest reading is that workload-level leakage is *weak*, not that it is below chance.

Why the gap? Most queries in W1–W3 do not directly probe the target row; they are diluted across the $k = 50$ -query workload. The classifier learns to identify the few informative queries, but the signal-to-noise ratio is governed by the per-query ϵ_q , not the cumulative budget. This empirical finding is consistent with prior work on amplification by sub-sampling and demonstrates that *cumulative-budget bounds are loose for realistic workload MIA*.

Cross-check with the R2 model. The R2 model predicts an upper bound on attacker advantage via the cumulative ϵ consumed. The empirical AUC falls well below this bound in all 32 cells, confirming that the model is conservative for leakage prediction (it overestimates risk, which is the safe direction). A tighter joint model of budget $\rightarrow$ attack success is left as future work.

10 Predictive Budget Allocator: Putting the Model to Work

The analytical model of §4 is descriptive: given a workload distribution, it predicts how much budget will be consumed. In this section I close the loop and use the same model *prescriptively*, as the backbone of a new execution mode that adapts the per-query privacy parameter ϵ_q online instead of fixing it offline.

Mechanism. At each incoming query, the allocator maintains an empirical frequency distribution over observed template hashes (Laplace-smoothed). After a small warmup phase (3–10% of the workload, used to seed the prior), it estimates the workload’s total unique-template count as $\hat{U} = \mathbb{E}[u_{k_{\text{total}}}]$ under $\hat{p}$ from Proposition 1, and allocates

$$\epsilon_q^{\text{pred}} = \frac{B}{\max(\hat{U}, 1)},$$

where B is the total privacy budget. The allocator caps the per-query value to the remaining budget so total spend never exceeds B by construction.

Algorithm 2 makes this decision explicit. Unlike Algorithm 1, whose per-query budget is a fixed input, here ϵ_q is a *runtime function* of the forecasted unique-template count $\hat{U}$: the non-trivial logic is the online re-estimation of $\hat{U}$ from the empirical template distribution and the resulting $B/\hat{U}$ allocation.

Why this is novel. PINQ [10], PrivateSQL [9], Chorus [8], and DOP-SQL [13] all expose a fixed per-query ϵ_q that the analyst chooses up front. None of them set ϵ_q from a forecast of *future workload-aggregate* consumption. Two systems do adapt budget in

Algorithm 2 Predictive per-query budget allocation

Require: Query q , total budget B , declared length k_{tot} , warmup size w , floor ϵ_0 , consumed c

Ensure: Per-query budget ϵ_q (0 = serve from cache or reject)

- 1: Append q 's template hash to history; update the unique-template set
- 2: **if** $B - c \leq \epsilon_0$ **then**
- 3: **return** 0 (budget exhausted; exact-repeat cache hits still served free)
- 4: **end if**
- 5: **if** queries seen $\leq w$ **then**
- 6: **return** $\max(\epsilon_0, \min(B/k_{\text{tot}}, B - c))$ (warmup: per-slot average under a flat prior)
- 7: **end if**
- 8: $\hat{p} \leftarrow$ Laplace-smoothed empirical template frequencies from history
- 9: $\hat{U} \leftarrow \mathbb{E}[u_{k_{\text{tot}}}]$ under $\hat{p}$ (Proposition 1)
- 10: **return** $\max(\epsilon_0, \min(B/\max(\hat{U}, 1), B - c))$ (offline-optimal target $\epsilon_q^* = B/u_k$)

adjacent senses: APEX [24] translates a per-query accuracy bound into the minimum-privacy ϵ for the *current* query at runtime, and PrivateSQL [9] allocates budget across views *offline* over a representative workload. The distinction here is that the predictive allocator chooses ϵ_q from an analytical forecast of future unique-template consumption (Proposition 1), rather than from the current query's accuracy target or a static per-view split. Its mathematical justification rests directly on Proposition 2: the optimal offline allocation is $\epsilon_q^* = B/u_k$, and the predictive allocator approximates u_k online.

Empirical comparison. Table 6 compares three strategies on Zipf workloads ($k = 100$, $B = 10, 30$ trials per cell). Naive divides B evenly across k queries ($\epsilon_q = 0.1$). Workload-aware uses the same fixed $\epsilon_q = 0.1$ but only charges cache misses—it answers all 100 queries but never spends more than $\sim 0.7\epsilon$ of the budget. Predictive uses the model to choose ϵ_q on the fly.

Interpretation. The predictive allocator delivers a modest MAE reduction (5–17% lower than naive: 16.8% at $\alpha = 0$ down to 5.4% at $\alpha = 2$) at the cost of denying some queries when the budget runs out. The trade-off surface is visible: ϵ_q per release jumps from 0.1 to $\approx 1.43 = B/m$, matching the theoretical optimum that the model derives from the predicted $\hat{U} \approx m$. Cache hits inherit whichever noisy answer was released at the time, so the cache becomes “locked in” to the early-release quality. To realize the full B/U utility benefit, the mechanism must *re-release* cached answers when the budget shows surplus—an extension I discuss in §11.

The gap across budget regimes. The MAE improvement at $B = 10$ is not a one-off. Sweeping the total budget over $B \in \{1, 5, 10, 50\}$ at $\alpha = 1$, $k = 100$ (Table 7) shows MAE reductions of 32% ($B=1$), $\sim 0.5\%$ ($B=5$), 18% ($B=10$), and 21% ($B=50$) relative to naive. The reduction is non-uniform: it is largest in the tightest-budget regime ($B=1$, where every release matters) and is negligible at $B=5$ (consistent with the near-identical naive/predictive entries in Table 7).

Scale invariance. The allocator's advantage persists at TPC-H SF=10 (60M lineitem rows, 20 trials, $\alpha = 1$, $k = 100$, $B = 10$): predictive cuts MAE to 6.03 vs naive's 10.01, a 40% reduction at $10\times$ the data volume. This is consistent with the model being structural, not data-dependent.

Composing with the temporal regime. The two new mechanisms compose. Combining the predictive allocator with a finite staleness tolerance τ (Section 5) yields lower MAE than temporal alone at every τ I tested (Table 8), in exchange for spending the full budget instead of the temporal-alone fraction. The choice between them is a deployment preference: *cheap re-noising with higher noise* (temporal alone) versus *full-budget spend with the lowest achievable noise* (combined).

11 Future Work

The analytical model and the predictive allocator open several concrete next steps, all of which were considered out of scope for the course-project version of this work and are deferred here.

Cache re-release on budget surplus. The predictive allocator's main weakness is that early-release cache entries lock in the noise scale used at the time. If the workload turns out more skewed than initially predicted, late-stage budget surplus cannot be used to refresh those entries. A re-release policy that periodically reruns the noise mechanism on stale cache entries when surplus exceeds a threshold would close this gap. The privacy accounting is straightforward (each re-release is a fresh ϵ_q -DP release).

Advanced composition with Rényi DP. My budget ledger uses basic sequential composition: $\epsilon_{\text{total}} = \sum \epsilon_i$. Switching to Rényi DP [11] or zCDP [34] would tighten the bound on the total privacy loss, allowing more releases (or larger per-query ϵ_q) under the same final (ϵ, δ) guarantee. The R2 model would then predict $\sum \epsilon_i$ rather than count u_k , with the closed-form structure mostly unchanged.

Burstiness and renewal-process arrival models. The i.i.d. template sampling assumption in §4 is a useful first approximation. Real workloads exhibit bursts: a dashboard refreshes 30 times in a minute, then sleeps for an hour. Replacing the Poisson assumption with a renewal process (Pareto or hyperexponential inter-arrival distribution) would let the temporal extension capture these patterns while preserving the closed-form $\mathbb{E}[\epsilon]$ derivation.

Real-world workload traces. The Zipf parametrization in this paper is a controlled sweep, not a measurement of real analyst behaviour. Replicating the experiments against a production-style query log (e.g., the Snowflake or Redshift trace anonymizations published in recent benchmark papers) would validate the model under genuinely heavy-tailed real workloads.

Cross-checking the model with leakage bounds. §9 showed that the cumulative- ϵ upper bound is loose for shadow-model MIA on realistic workloads. A tighter joint bound that uses the workload's $\mathbb{E}[u_k]$ to discount the per-target privacy loss is a clear next theoretical step.

Equivalence-checked semantic cache. The semantic L2 cache of §8 fails because tree-kernel and embedding similarity do not imply alpha-equivalence. Pairing those similarity tools with a symbolic

Table 6: Predictive allocator vs fixed- ϵ_q baselines (mean over 30 trials, $k = 100$, total budget $B = 10$, warmup fraction 3%).

α	Mode	ϵ spent	MAE	Queries answered	ϵ_q per release
0.0	NAIVE	10.0	9.86	100/100	0.10
	WORKLOAD-DP	0.7	9.46	100/100	0.10 (per miss)
	PREDICTIVE	10.0	8.21	69/100	1.43 (per miss)
1.0	NAIVE	10.0	9.81	100/100	0.10
	WORKLOAD-DP	0.7	9.96	100/100	0.10 (per miss)
	PREDICTIVE	10.0	8.98	78/100	1.43 (per miss)
2.0	NAIVE	10.0	9.96	100/100	0.10
	WORKLOAD-DP	0.7	10.22	100/100	0.10 (per miss)
	PREDICTIVE	10.0	9.42	88/100	1.43 (per miss)

Table 7: Predictive allocator vs baselines across budget regimes. 20 trials per cell.

B	Mode	MAE	ϵ spent	Queries answered
1	NAIVE	101.1	0.99	99/100
	WORKLOAD	87.9	0.07	100/100
	PREDICTIVE	68.7	1.00	80/100
5	NAIVE	20.4	5.0	100/100
	WORKLOAD	23.7	0.35	100/100
	PREDICTIVE	20.3	5.0	76/100
50	NAIVE	2.0	50.0	100/100
	WORKLOAD	1.93	3.5	100/100
	PREDICTIVE	1.58	50.0	84/100

Table 8: Predictive + temporal staleness τ vs temporal alone. $\alpha = 1$, $k = 100$, $B = 50$, 20 trials.

τ	TEMPORAL alone		TEMPORAL + PREDICTIVE	
	MAE	ϵ	MAE	ϵ
10	2.11	19.0	2.08	50.0
25	2.03	11.3	1.72	50.0
50	2.09	6.7	1.70	50.0
100	1.76	3.5	1.25	50.0
1000	1.80	3.5	1.74	50.0

equivalence prover (predicate normalization, range-merging, etc.) would let the system safely admit a semantic match only when the queries are provably equivalent, preserving the post-processing property.

Multi-table and JOIN support. The current scope (single-table aggregates) was deliberately approved at the milestone stage. The model’s framework—occupancy over a template distribution—generalizes naturally to JOIN-shaped queries if a sensitivity bound (residual sensitivity [35] or R2T [36]) is available per template. Incorporating join sensitivity is the most direct path to a thesis-grade extension.

Decomposed AVG with per-group accounting. The current implementation collapses AVG to a single Laplace release with sensitivity B_c (worst case: group of size one). A SUM+COUNT decomposition would charge $2\epsilon_q$ per group but apply noise of scale B_c/ϵ_q to the SUM and $1/\epsilon_q$ to the COUNT, dividing through to yield mean-error

roughly $O(B_c/(n\epsilon_q))$ for groups of size n . For TPC-H scales where most groups have $n \gg 1$ (e.g., l_returnflag groups in lineitem), this is a measurable improvement. The change touches only the analyzer and mechanism layer; the budget ledger and predictive allocator already support multi-aggregate sequential composition.

12 Discussion and Conclusion

What I found. The campaign produced four findings worth stating explicitly.

- (1) **Workload-aware budget consumption is a function of $\mathbb{E}[u_k]$, not the cache mechanism.** The closed form $\mathbb{E}[\epsilon_{wa}] = \epsilon_q \cdot \mathbb{E}[u_k]$ holds with $< 3\%$ relative error in 21 of 24 main-grid cells, holds *exactly* on a constructed mixed W1+W4 workload (Table 3), and is invariant to TPC-H scale (SF=1 vs SF=10 agree within 0.03 units) and to ϵ_q (which cancels structurally). This means a deployment can estimate its privacy cost *before* running any queries from the workload’s template distribution alone.
- (2) **The toy $1 - 1/k$ rule is a corner case.** The milestone identified $1 - 1/k$ for perfect repetition; the model places this at Limit A ($\alpha \rightarrow \infty$) and predicts the full interior between it and naive composition (Limit B). At $\alpha = 2$, $k = 200$, $m = 7$ the savings ratio is empirically 96.5% ($\mathbb{E}[u_k] = 6.93$, $S = 1 - 6.93/200$) and the model predicts 96.5% ($\mathbb{E}[u_k] = 6.90$).
- (3) **The cumulative- ϵ bound is loose for realistic multi-query MIA.** Shadow-model MIA AUC across W1–W4 ranges from 0.14 to 0.58, well below the cumulative bound $e^{\sum \epsilon_i} / (1 + e^{\sum \epsilon_i}) \rightarrow 1.0$ that current accounting assumes. Practically: privacy budget is overspent if the threat model assumes worst-case observation.
- (4) **Semantic similarity is not equivalence.** Tree-kernel + embedding matching admits dangerous false positives (queries with different true answers but similar ASTs) and misses textbook equivalences (commutative AND, integer comparators). The cache is correct only when paired with a symbolic equivalence prover; without one, "semantic DP cache" is not actually DP.

Where the model wins, where it loses. The model gives its largest savings on skewed workloads (W1, W2-parametric): savings reach $1 - 1/k$ in the repetitive limit and $\approx 1 - \mathbb{E}[u_k]/k$ in the Zipf- $\alpha=1$ regime, with predictions within 3% of the empirical mean across 21

of 24 main-grid cells. The model also identifies failure regimes: on uniform workloads ($\alpha \rightarrow 0$) and drill-down sequences (W4), every query is structurally unique, the savings ratio collapses to zero, and the reconstruction attack succeeds at any practical ϵ . The temporal extension makes the cost of freshness explicit: a dashboard with staleness tolerance $\tau=10$ over a horizon $T=100$ pays roughly an order of magnitude more budget than the static-snapshot case (the model gives an upper-bound prediction, exact magnitudes depend on the realized arrival sequence), recovering the milestone's exact-repeat assumption only at $\tau \rightarrow \infty$.

Two assumptions remain to be relaxed. First, template sampling is i.i.d.; real workloads burst (dashboards refresh in short volleys, then sleep). Second, the temporal extension uses Poisson updates with a single invalidation probability q ; production systems have correlated invalidations. Both are tractable in the same closed-form framework and are listed in Section 11. With those caveats, the report contributes (i) a closed-form workload-budget model that recovers the milestone's $1 - 1/k$ rule as a corner case and predicts the full Zipf interior; (ii) a temporal extension that couples budget to data freshness; (iii) a predictive allocator that targets the analytical optimum $\epsilon_q^* = B/u_k$; and (iv) a leakage analysis that matches the theoretical MIA and reconstruction bounds in the tight regimes and exposes the slack in the cumulative- ϵ bound elsewhere. The middleware, the full 4,155-trial experimental corpus, and the figures are reproducible from the repository.

References

- [1] D. E. Denning. 1979. The Tracker: A Threat to Statistical Database Security. *ACM Transactions on Database Systems* 4, 1, 76–96.
- [2] I. Dinur and K. Nissim. 2003. Revealing Information while Preserving Privacy. In *PODS*, 202–210.
- [3] C. Dwork, F. McSherry, K. Nissim, and A. Smith. 2006. Calibrating Noise to Sensitivity in Private Data Analysis. In *TCC*, 265–284.
- [4] C. Dwork, G. N. Rothblum, and S. Vadhan. 2010. Boosting and Differential Privacy. In *FOCS*, 51–60.
- [5] C. Dwork and A. Roth. 2014. *The Algorithmic Foundations of Differential Privacy*. F&T in Theor. CS 9, 3–4, 211–407.
- [6] P. Flajolet, D. Gardy, and L. Thimonier. 1992. Birthday Paradox, Coupon Collectors, Caching Algorithms and Self-Organizing Search. *Discrete Applied Mathematics* 39, 3, 207–229.
- [7] N. Johnson, J. P. Near, and D. Song. 2018. Towards Practical Differential Privacy for SQL Queries. *PVLDB* 11, 5, 526–539.
- [8] N. Johnson, J. P. Near, J. Hellerstein, and D. Song. 2020. Chorus: A Programming Framework for Building Scalable Differential Privacy Mechanisms. In *EuroS&P*.
- [9] I. Kotsogiannis, Y. Tao, X. He, M. Fanaeepour, A. Machanavajjhala, M. Hay, and G. Miklau. 2019. PrivateSQL: A Differentially Private SQL Query Engine. *PVLDB* 12, 11, 1371–1384.
- [10] F. McSherry. 2009. Privacy Integrated Queries: An Extensible Platform for Privacy-preserving Data Analysis. In *SIGMOD*, 19–30.
- [11] I. Mironov. 2017. Rényi Differential Privacy. In *CSF*, 263–275.
- [12] NIST. 2025. Guidelines for Evaluating Differential Privacy Guarantees. *NIST SP 800-226*.
- [13] J. Yu, W. Dong, J. Fang, D. Sun, and K. Yi. 2024. DOP-SQL: A General-purpose, High-utility, and Extensible Private SQL System. *PVLDB* 17, 12, 4385–4388.
- [14] W. Dong, D. Sun, and K. Yi. 2023. Better than Composition: How to Answer Multiple Relational Queries under DP. *PACMMOD* 1, 2.
- [15] M. Collins and N. Duffy. 2001. Convolution Kernels for Natural Language. In *NeurIPS*, 625–632.
- [16] A. Moschitti. 2006. Making Tree Kernels Practical for Natural Language Learning. In *EACL*, 113–120.
- [17] Z. Feng et al. 2020. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In *EMNLP Findings*, 1536–1547.
- [18] D. Guo et al. 2021. GraphCodeBERT: Pre-training Code Representations with Data Flow. In *ICLR*.
- [19] C. Li, M. Hay, V. Rastogi, G. Miklau, and A. McGregor. 2010. Optimizing Linear Counting Queries Under Differential Privacy. In *PODS*, 123–134.
- [20] R. McKenna, G. Miklau, M. Hay, and A. Machanavajjhala. 2018. Optimizing Error of High-Dimensional Statistical Queries Under Differential Privacy. *PVLDB* 11, 10, 1206–1219.
- [21] M. Hardt, K. Ligett, and F. McSherry. 2012. A Simple and Practical Algorithm for Differentially Private Data Release. In *NeurIPS*, 2339–2347.
- [22] A. Roth and T. Roughgarden. 2010. Interactive Privacy via the Median Mechanism. In *STOC*.
- [23] M. Hardt and G. N. Rothblum. 2010. A Multiplicative Weights Mechanism for Privacy-Preserving Data Analysis. In *FOCS*, 61–70.
- [24] C. Ge, X. He, I. F. Ilyas, and A. Machanavajjhala. 2019. APEX: Accuracy-Aware Differentially Private Data Exploration. In *SIGMOD*.
- [25] I. Kotsogiannis, A. Machanavajjhala, M. Hay, and G. Miklau. 2017. Pythia: Data Dependent Differentially Private Algorithm Selection. In *SIGMOD*.
- [26] S. Berghel et al. 2022. Tumult Analytics: a robust, easy-to-use, scalable, and expressive framework for differential privacy. arXiv:2212.04133.
- [27] OpenDP Project and SmartNoise. 2023. OpenDP Library and SmartNoise SQL. <https://opendp.org/>.
- [28] R. J. Wilson, C. Y. Zhang, W. Lam, D. Desfontaines, D. Simmons-Marengo, and B. Gipson. 2020. Differentially Private SQL with Bounded User Contribution. *PoPETs* 2020, 2, 230–250.
- [29] S. Zhang and X. He. 2023. DProvDB: Differentially Private Query Processing with Multi-Analyst Provenance. *Proc. ACM Manag. Data* 1, 4, Article 267 (presented at SIGMOD 2024). arXiv:2309.10240.
- [30] B. Zhang, V. Doroshenko, P. Kairouz, T. Steinke, A. Thakurta, Z. Ma, E. Cohen, H. Apte, and J. Spacek. 2024. Differentially Private Stream Processing at Scale (DP-SQLP). *PVLDB* 17, 12, arXiv:2303.18086.
- [31] N. Grislain, P. Roussel, and V. de Sainte Agathe (Sarut Technologies). 2024. Qrlew: Rewriting SQL into Differentially Private SQL. arXiv:2401.06273. Open-source: <https://github.com/Qrlew/qrlew>.
- [32] T. Matsumoto, S. Takakura, S. Takagi, and S. Hasegawa. 2026. DPSQL+: A Differentially Private SQL Library with a Minimum Frequency Rule. arXiv:2602.22699.
- [33] M. Hay, A. Machanavajjhala, G. Miklau, Y. Chen, and D. Zhang. 2016. Principled Evaluation of Differentially Private Algorithms using DPBench. In *SIGMOD*.
- [34] M. Bun and T. Steinke. 2016. Concentrated Differential Privacy: Simplifications, Extensions, and Lower Bounds. In *TCC*, 635–658.
- [35] W. Dong and K. Yi. 2021. Residual Sensitivity for Differentially Private Multi-Way Joins. In *SIGMOD*, 432–445.
- [36] W. Dong, J. Fang, K. Yi, Y. Tao, and A. Machanavajjhala. 2022. R2T: Instance-optimal Truncation for Differentially Private Query Evaluation with Foreign Keys. In *SIGMOD*, 759–772.
- [37] K. Kostopoulou, P. Tholoniati, A. Cidon, R. Geambasu, and M. Lécuyer. 2023. Turbo: Effective Caching in Differentially-Private Databases. In *SOSP*. arXiv:2306.16163.
- [38] M. Mazumdar, T. Humphries, J. Liu, M. Rafuse, and X. He. 2023. Cache Me If You Can: Accuracy-Aware Inference Engine for Differentially Private Data Exploration. *PVLDB* 16, 4, arXiv:2211.15732.
- [39] C. Dwork, M. Naor, T. Pitassi, and G. N. Rothblum. 2010. Differential Privacy Under Continual Observation. In *STOC*, 715–724.